

RS003 Week 13 — English Worksheet

Topic: Many Cooperating Functions · **Course:** Pygame Space Shooter · **Time:** about 45 minutes

This week you split your game into **several small functions** — one for the starfield, one for the ship, one for an enemy, one for the aiming laser. The functions work together by sharing the ship's X position, and the laser changes colour when the ship lines up under the enemy.

 Words to know: **function**, **call**, **share a value**, **enemy**, **laser**

1 · Predict

Read each piece of code. Before you run it, write what you think will happen.

```
def draw_stars():
    for i in range(40):
        pygame.draw.circle(screen, WHITE, (i * 20 + 10, 30), 2)

def draw_ship(ship_x):
    ...

draw_stars()
draw_ship(CENTER_X)
```

draw_stars takes no input but **draw_ship** takes a **ship_x**. Why does only one of them need a value?


```
def draw_laser(ship_x):
    distance = abs(ship_x - ENEMY_X)
    color = YELLOW if distance < 20 else RED
    pygame.draw.line(screen, color, (ship_x, HEIGHT - 120), (ship_x, ENEMY_Y), 4)
```

`abs(ship_x - ENEMY_X)` is how far the ship is from the enemy. When does the laser turn yellow?


```
draw_stars()
draw_enemy()
draw_ship(ship_x)
draw_laser(ship_x)
```

Four functions are called in a row. Which one is drawn last, and what does that mean for what sits in front?

2 · Run It

 **Type the example, run it, and line up the enemy**

```
import pygame
pygame.init()

WIDTH, HEIGHT = 800, 600
CENTER_X = WIDTH // 2
screen = pygame.display.set_mode((WIDTH, HEIGHT))

SPACE = (10, 12, 40)
CYAN = (80, 220, 255)
WHITE = (255, 255, 255)
RED = (230, 60, 60)
YELLOW = (255, 210, 40)

ENEMY_X = 600
ENEMY_Y = 120

def draw_stars():
    for i in range(40):
        pygame.draw.circle(screen, WHITE, (i * 20 + 10, 30), 2)

def draw_ship(ship_x):
    pygame.draw.rect(screen, CYAN, (ship_x - 20, HEIGHT - 120, 40, 50))
    pygame.draw.polygon(screen, WHITE, [(ship_x, HEIGHT - 140), (ship_x - 20,
HEIGHT - 120), (ship_x + 20, HEIGHT - 120)])

def draw_enemy():
    pygame.draw.rect(screen, RED, (ENEMY_X - 18, ENEMY_Y - 18, 36, 36))

def draw_laser(ship_x):
    distance = abs(ship_x - ENEMY_X)
    color = YELLOW if distance < 20 else RED
    pygame.draw.line(screen, color, (ship_x, HEIGHT - 140), (ship_x, ENEMY_Y), 4)

ship_x = CENTER_X
SPEED = 5
MIN_X, MAX_X = 100, WIDTH - 100

clock = pygame.time.Clock()
running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False
    keys = pygame.key.get_pressed()
    if keys[pygame.K_LEFT] and ship_x > MIN_X:
```

```
        ship_x -= SPEED
    if keys[pygame.K_RIGHT] and ship_x < MAX_X:
        ship_x += SPEED

    screen.fill(SPACE)
    draw_stars()
    draw_enemy()
    draw_laser(ship_x)
    draw_ship(ship_x)
    pygame.display.flip()
    clock.tick(60)

pygame.quit()
```

Fly slowly under the enemy. What happens to the laser colour as you line up?

3 · Spot the Bug 🐛

Each block below was meant to do something but is broken. Fix it, then explain why the original was wrong.

Bug A — The laser should follow the ship, but it is stuck in the centre and never moves.

```
def draw_laser():  
    pygame.draw.line(screen, RED, (CENTER_X, HEIGHT - 140), (CENTER_X, ENEMY_Y), 4)
```

Hint: the laser needs the ship's position passed in.

Write the fixed code (the whole function):

Why was it wrong? Why does your fix work?

Bug B — The ship should sit **in front of** the laser, but right now the laser is drawn last and covers the ship.

```
draw_stars()  
draw_enemy()  
draw_ship(ship_x)  
draw_laser(ship_x)
```

Hint: whatever is drawn last sits on top.

Write the fixed code (calls in the right order):

Why was it wrong? Why does your fix work?

Bug C — The laser is supposed to turn yellow only when the ship is **close** to lined up, but it is yellow everywhere.

```
distance = ship_x - ENEMY_X
color = YELLOW if distance < 20 else RED
```

Hint: without `abs`, a big negative distance is also "less than 20".

Write the fixed code:

Why was it wrong? Why does your fix work?

4 · Modify It

Move the enemy somewhere new

Change `ENEMY_X` so the enemy sits in a different spot, then line up under it.

Write the new ENEMY_X you chose:

Add a fourth function of your own

Write one more small function — for example `draw_score()` or `draw_meteors()` — and call it in the loop.

Write your function name and what it draws:

Hint: a function is just a name for a group of drawing lines. If it needs a changing value (like the ship position), pass it in as a parameter.

5 · Make It 📷

Build a game split into functions

Build a program where:

1. the starfield, ship, enemy, and laser are each their own function,
2. the functions share the ship's position,
3. the laser changes colour when you line up under the enemy.

Send a **photo or video** of your lined-up laser, then explain what you did. Use these sentence starters — write 4 to 6 sentences total.

First, I split my code into functions for ...

The functions share the ship position by ...

The laser turns yellow when ...

I used `abs(...)` to ...

One tricky moment was when ...

If I had more time, I would ...

6 · Record Your Walkthrough 🎥

Take a video while you line up under the enemy. Talk through it like you are teaching someone. Try to use these words: **function**, **call**, **share**, **enemy**, **laser**.

1. Show the laser turning yellow under the enemy.
2. Read your function names out loud and say what each one draws.
3. Explain how the ship position is shared between functions.
4. Point at the line that decides the laser colour.

Write what you will say in your video. Plan it here before you record.

Submit ✓

Send this worksheet + your photo or video to teacher on KakaoTalk.